



香港大學

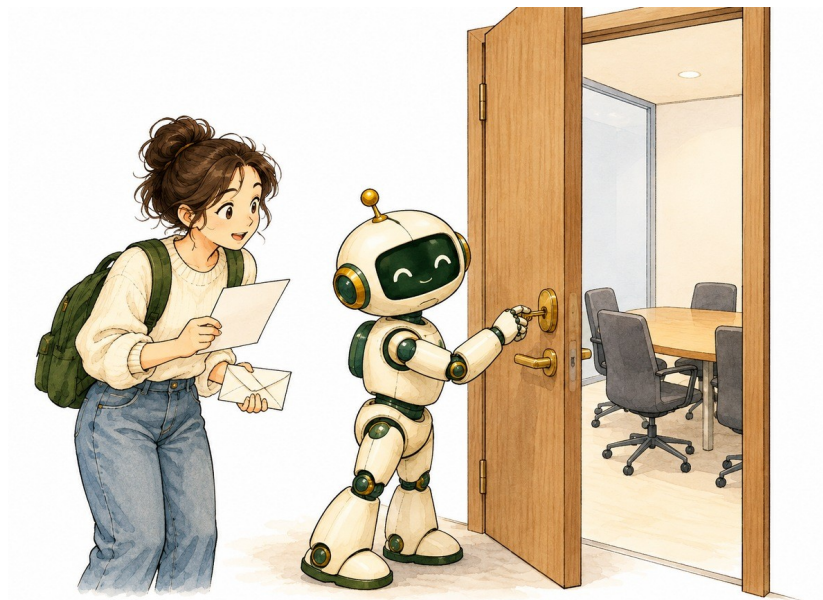
THE UNIVERSITY OF HONG KONG



Agents

Planning · Memory
Tool-use

COMP7607A · LECTURE 5



Lecture outline

01 Overview

02 Architecture: a model inside a loop

03 Planning: choose the next step

04 Memory: preserve what matters

05 Tool-use: make actions executable

06 Insights: challenges over longer tasks

07 Assignment: build and improve a policy

Today: book a classroom

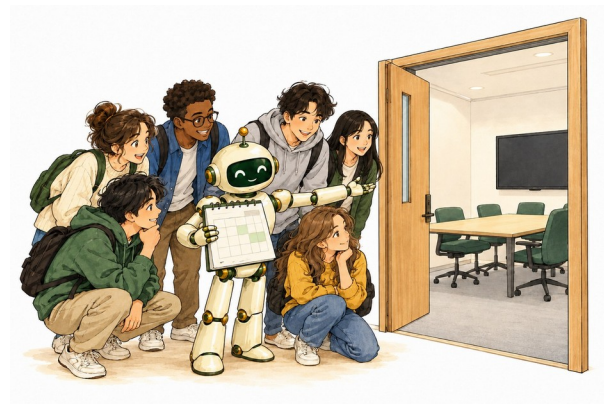
“Book a room and invite our group.”

6 students

Friday, 14:00–18:00

30 minutes

Step-free access



Reserve the room. Then send invitations to all six.

The booking task, step by step

01

Find rooms
for 6 people



02

Check each
room's times



03

Book the
room
and verify



04

Send the
invitations

Which rooms can hold six students?

[Find rooms](#)[Check times](#)[Book + verify](#)[Send invites](#)

ROOM CATALOGUE

Room A

4 seats

Too small

Room C

8 seats

Step-free

Room D

10 seats

Step-free

Rooms C and D meet the room requirements.

Check Availability

Find rooms

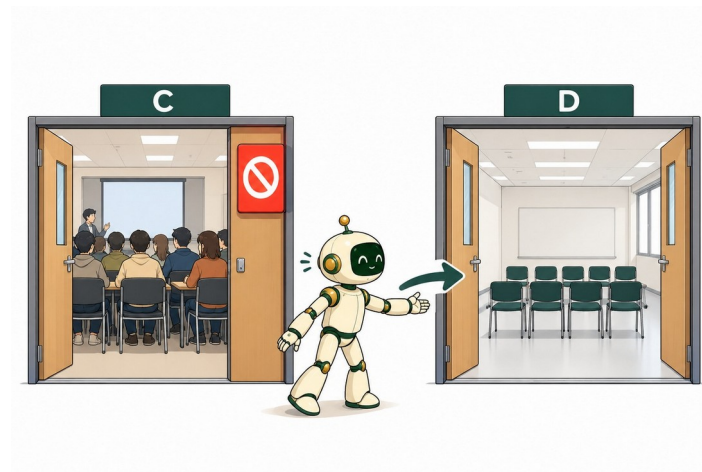
Check times

Book + verify

Send invites

Room C · 14:30–15:00
Occupied

Room D · 16:00–16:30
Available



Book Room D and check the result

Find rooms

Check times

Book + verify

Send invites

```
reserve_room("D", "16:00", 30)
```

Confirmed
BK-1042
16:00–16:30



Send invitations after the booking is confirmed

Find rooms

Check times

Book + verify

Send invites

CONFIRMED BOOKING

Room D
Friday 20 Nov
16:00–16:30



TOOL RESULT

```
id = "BK-1042"  
send_invites(id, group)  
  
status: "sent"  
sent_to: 6  
message_id: "INV-1042"
```

Why planning, memory and tools?

Planning

**Which room
should we try?**

Memory

**What have we
already done?**

Tools

**How do we
book and send?**

Each result can change the next action.

Feedback loops and ReAct

1948 Cybernetics



Human act, sense and adjust.

Source: Wiener (1948) / Yao et al., ReAct (2022; ICLR 2023)

2022 ReAct

ReAct: SYNERGIZING REASONING AND ACTING IN LANGUAGE MODELS

Shunya Yao¹, Jeffrey Zhao², Dian Yu¹, Nan Du¹, Ishak Shafiq², Karthik Narasimhan¹, Yuntao Cao²

¹Department of Computer Science, Princeton University

²Google Research, Brain team

³shunya@princeton.edu, karthikn@princeton.edu, yuntao@princeton.edu

ABSTRACT

While large language models (LLMs) have demonstrated impressive performance across tasks in language understanding and interactive decision making, their abilities for reasoning (e.g. chain-of-thought prompting) and acting (e.g. action plan generation) have primarily been studied in separate topics. In this paper, we explore the use of LLMs to generate both reasoning traces and task-specific actions as an interleaved manner, allowing for greater synergy between the two. Reasoning traces help the model induce, track, and update action plans as well as handle exceptions, while action chains (e.g. tool-use) with and gather additional information from external sources such as knowledge bases or environments. We apply our approach, named *ReAct*, to a diverse set of language and decision-making tasks and demonstrate its effectiveness over state-of-the-art baselines in improving human interpretability and transparency. Concretely, on question answering (HotpotQA) and fact verification (FactV), *ReAct* overcomes prevalent issues of hallucination and error propagation in chain-of-thought reasoning by interacting with a single Wikipedia API, and generating human-like task-solving trajectories that are more interpretable than baselines without reasoning traces. Furthermore, on two interactive decision-making benchmarks (ALFWorld and WebShop), *ReAct* outperforms imitation and reinforcement learning methods by an absolute success rate of 4% and 10% respectively, while being prompted with only one or two in-context examples.

1 INTRODUCTION

A unique feature of human intelligence is the ability to seamlessly combine task-oriented actions with verbal reasoning in two spaces (Schmidhuber & Weng, 2015), which has been theorized to play an important role in human cognitive functions like navigation or organization (Vijayakumar & Schaal, 2000; Forthmann & 2018) and maintaining a working memory (Buckley & 2015). Consider the example of cooking up a dish in the kitchen. Between any two specific actions, we may reason in language in order to keep progress: “Now that everything is set, I should heat up the pot of water”, to handle exceptions or adjust the plan according to the situation: “I don’t have salt, so let me use my sauce and pepper instead”, and to realize when external information is needed: “How do I prepare dough? Let me search on the Internet”. We may also not open a cookbook to read the recipe, open the fridge, check ingredients to support our reasoning and to answer questions: “What shall I eat today after lunch?”. This tight synergy between “acting” and “reasoning” allows humans to learn new tasks quickly and perform robust decision making or reasoning even under previously unseen circumstances or facing information uncertainties.

Recent results have hinted at the possibility of combining verbal reasoning with interactive decision making in autonomous systems. On one hand, properly prompted large language models (LLMs) have demonstrated emergent capabilities to carry out several steps of reasoning traces to derive

¹Work done while intern at Google Brain.

arXiv:2210.03629v1 [cs.CL] 6 Oct 2022

LLMs reason, act and observe.



香港大學
THE UNIVERSITY OF HONG KONG

AGENTS · LECTURE 5

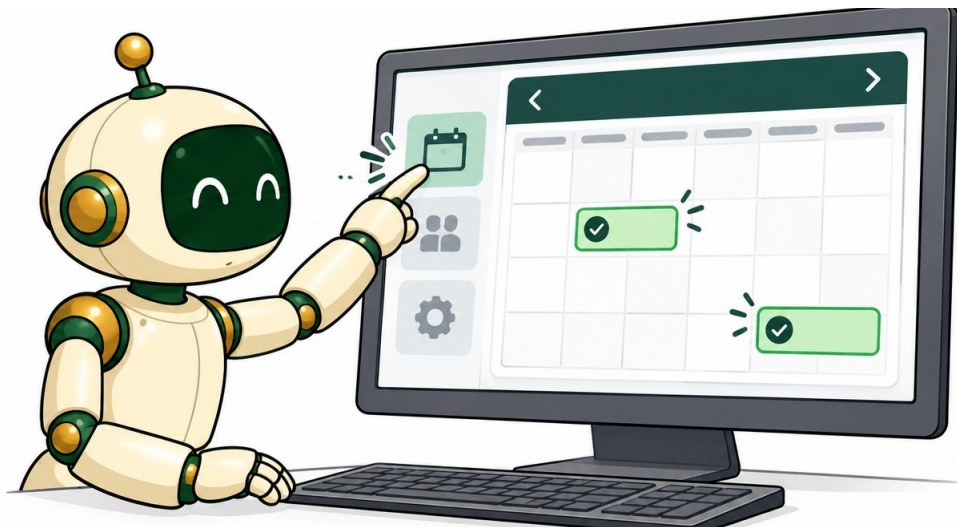
02

Agent architecture

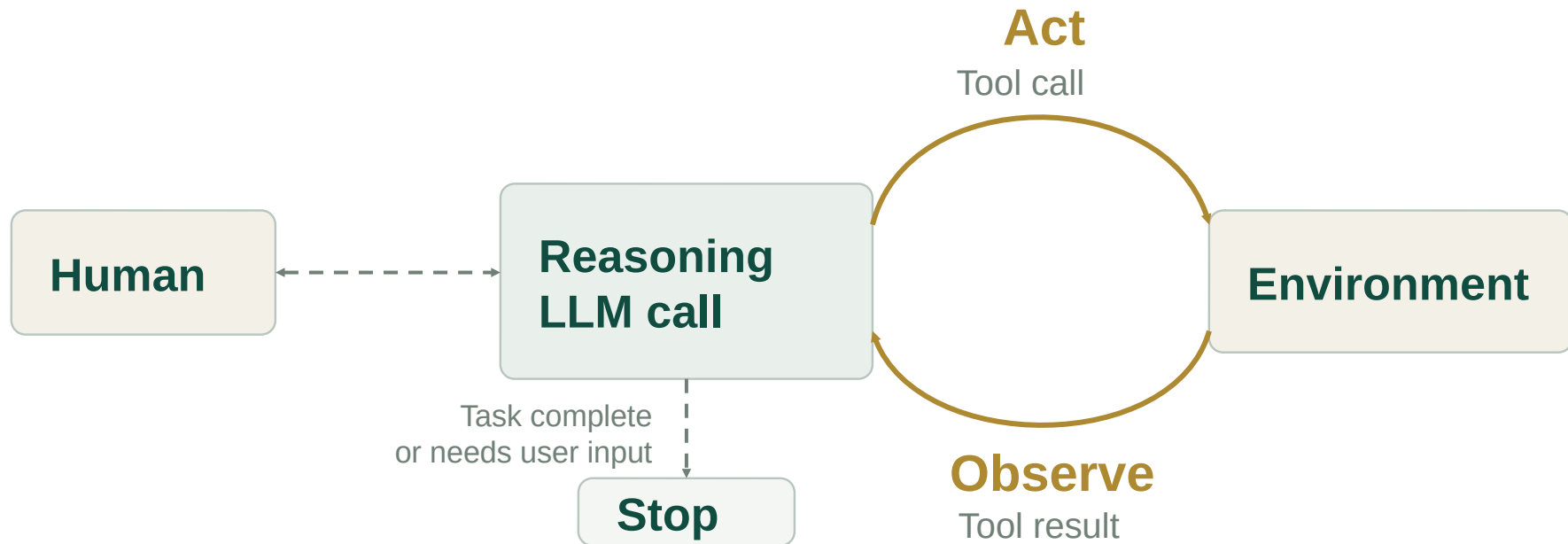
SO FAR

We need decisions and actions
across several steps.

PART 2 OF 7



ReAct Loop: Reasoning, Act, Observe



Reasoning: what should we check first?

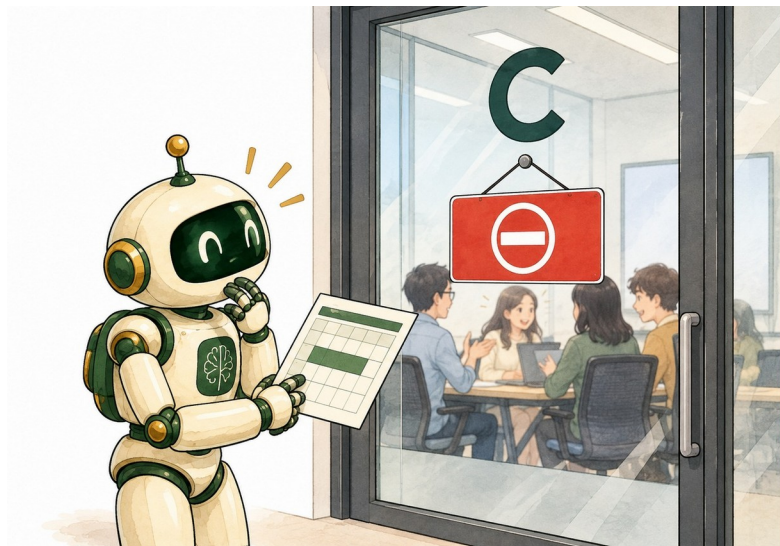
Reasoning

Act

Observe

Room C fits the request.

Is it free at 14:30?



Act: checking the booking system

Reasoning

Act

Observe

Check Room C

20 Nov · 14:30 · 30 min

```
check_room("C", "2026-11-20T14:30+08:00", 30)
```

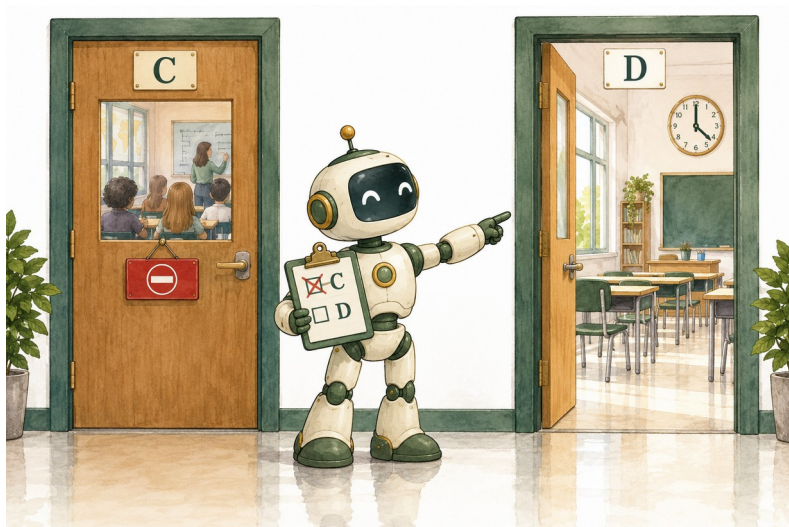
The harness runs the tool.

Observe: Room C is occupied

Reasoning

Act

Observe



C is occupied.

Next reasoning step

Check D at 16:00.

ReAct: reasoning uses feedback to guide action

User allows 14:00–18:00. The group needs 30 minutes.

Reasoning

Act

Observe

Try C at 14:30.

14:30–15:00 fits
the user's time window.



check_room(C)



Occupied



Try D at 16:00.

C is busy. 16:00–16:30
also fits the window.



check_room(D)



Available

The first observation changes the second action.

Context is what the model receives

User Message

System Prompt

Memory

Tool List

Tool Result

Book a room for six students.
Room C is occupied at 14:30.
Send invites after confirmation.



**Language
model**

User Message: this task's goal

User Message	System Prompt	Memory	Tool List	Tool Result
--------------	---------------	--------	-----------	-------------

USER MESSAGE

Book a step-free room for 6 students.
20 November 2026, 14:00-18:00.
We need it for 30 minutes.
Then send invitations to our group.

System Prompt: who the agent is and how it works

User Message

System Prompt

Memory

Tool List

Tool Result

SYSTEM PROMPT

```
You are a campus booking assistant.  
Help users find and reserve suitable rooms.  
Check availability before you book.  
Use tool results to confirm what happened.  
Ask the user when no valid option remains.  
Send invitations when the user requests them.
```

Different assistants need different system prompts

User Message

System Prompt

Memory

Tool List

Tool Result

USER

“Check my answer: $2x + 3 = 11$, $x = 7$.”

Learning tutor

SYSTEM PROMPT

You are a tutor.
Give a hint before the answer.

**“Subtract 3 first.
What is $2x$?”**

Practice grader

SYSTEM PROMPT

You grade practice answers.
Give the result and correct answer.

“Incorrect. $x = 4$.”

A vague request needs the right follow-up questions

User Message

System Prompt

Memory

Tool List

Tool Result

USER

“Book us a room tomorrow.”

No booking-specific instructions

The application gives no booking workflow.

“What kind of room do you need?”

Missing details still come from the user.

Booking System Prompt

You are a campus booking assistant.
Collect people, time and access needs.

**“How many people?
When, and for how long?
Any access needs?”**

Memory: a checklist in MEMORY.md

User Message

System Prompt

Memory

Tool List

Tool Result

MEMORY.md

```
[x] Find rooms for 6: C (8), D (10)
[x] Check C at 14:30: occupied
[ ] Check D at 16:00
[ ] Book a room and verify its receipt
[ ] Send invitations to all six
```

Tool List: room tools and general file tools

User Message

System Prompt

Memory

Tool List

Tool Result

ROOM TOOLS

find_rooms(...)

Find suitable rooms

check_room(...)

Check a room's available times

reserve_room(...)

Book and receive a confirmation

send_invites(...)

Send the confirmed details

GENERAL FILE TOOLS

read_file(path)

Read a note or memory file

write_file(path, content)

Create or replace a file

edit_file(path, old_text, new_text)

Update part of a file

list_dir(path)

Find files in a folder

Tool Result: what actually came back?

User Message	System Prompt	Memory	Tool List	Tool Result
--------------	---------------	--------	-----------	-------------

ASSISTANT · call_1

```
check_room(  
  room="C",  
  start="2026-11-20T14:30+08:00",  
  minutes=30  
)
```



TOOL RESULT · call_1

```
{  
  "available": false,  
  "reason": "occupied"  
}
```

The call ID connects the action to its observation.

The booking context the model sees

User Message

System Prompt

Memory

Tool List

Tool Result

SYSTEM PROMPT

You are a campus booking assistant.
Check results before confirming.

USER MESSAGE

6 people, 20 Nov, 14:00–18:00.
30 minutes, step-free. Book and invite.

MEMORY.md

```
[x] Found C and D
[x] C is occupied at 14:30
[ ] Check D, book, invite
```

TOOL LIST

```
find_rooms    check_room
reserve_room  send_invites
```

```
read_file(...)  write_file(...)
edit_file(...)  list_dir(...)
```

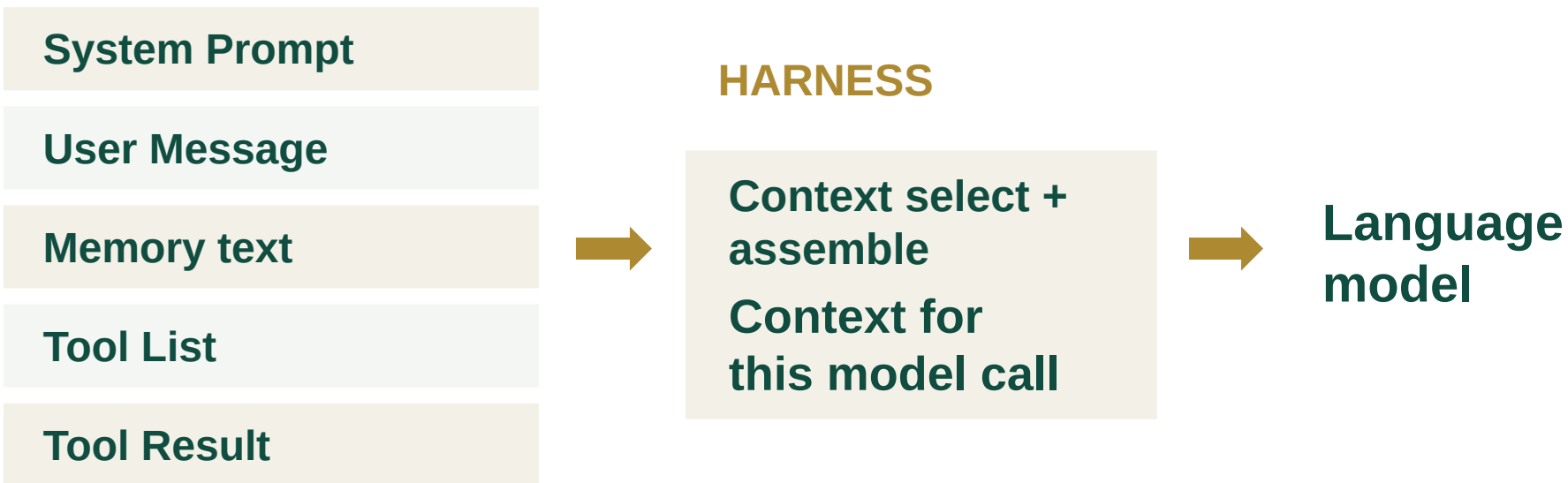
ACT · ASSISTANT TOOL CALL

```
check_room("C", "14:30", 30)
```

OBSERVE · TOOL RESULT

```
{"available": false}
```

Agent Harness: assembling the context



Building the next context

CURRENT CALL

System Prompt: campus booking assistant

User: 6 students, Friday, 30 minutes

Memory: Rooms C and D fit the group

Tool List: find / check / reserve / invite

Earlier messages: find suitable rooms

Assistant tool call

find_rooms(min_capacity=6)

Tool result

C: 8 seats D: 10 seats

Keep



NEXT CALL

System Prompt: campus booking assistant

User: 6 students, Friday, 30 minutes

Memory: Rooms C and D fit the group

Tool List: find / check / reserve / invite

Earlier messages: find suitable rooms

Assistant tool call

find_rooms(min_capacity=6)

Tool result

C: 8 seats D: 10 seats

New messages: check Room C

Assistant tool call

check_room("C", "14:30", 30)

Tool result

C is occupied at 14:30

Keep the history.
Append the new call and result.

The Basic Agent Loop: ReAct

next = the model's output: a tool call, a final reply, or a question.

PSEUDOCODE

```
while budget_remains():  
    context = build_context(state)  
    next = model(context)  
    if next.done or next.needs_user:  
        reply_to_user(next); break  
    result = run_allowed_tool(next.call)  
    state.add(next.call, result)
```

WHAT THIS STEP DOES

Stay within the budget

1 Assemble the messages

2 Ask the model

3 Finish, or ask the user

done: finished needs_user: ask

4 Run the tool in next.call

5 Add the call + result

Previous slide: keep check_room(C) + “C is occupied” for the next context.



香港大學
THE UNIVERSITY OF HONG KONG

AGENTS · LECTURE 5

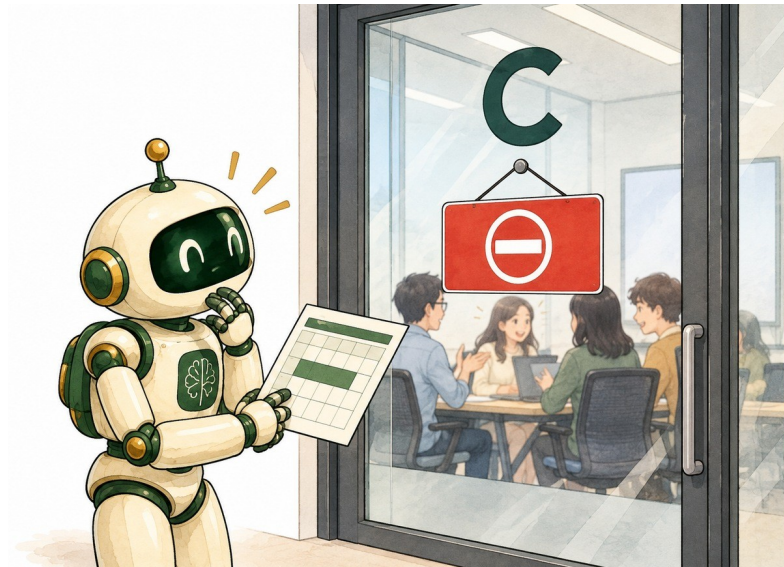
03

Planning

SO FAR

The loop can act and receive results.

PART 3 OF 7



OUR NEXT QUESTION

Which action moves us toward the goal?

Planning: a route to the finished task

01

Find rooms
for 6 people



02

Check each
room's times



03

Book the
room
and verify



04

Send the
invitations

No room is available?



Stop and ask the user.

A plan can change when the facts change.

The checklist changes as work gets done

AFTER CHECKING TIMES

- ☒ Find suitable rooms
- ☒ Check their times
- ☐ Book and verify
- ☐ Send invitations



AFTER BOOKING

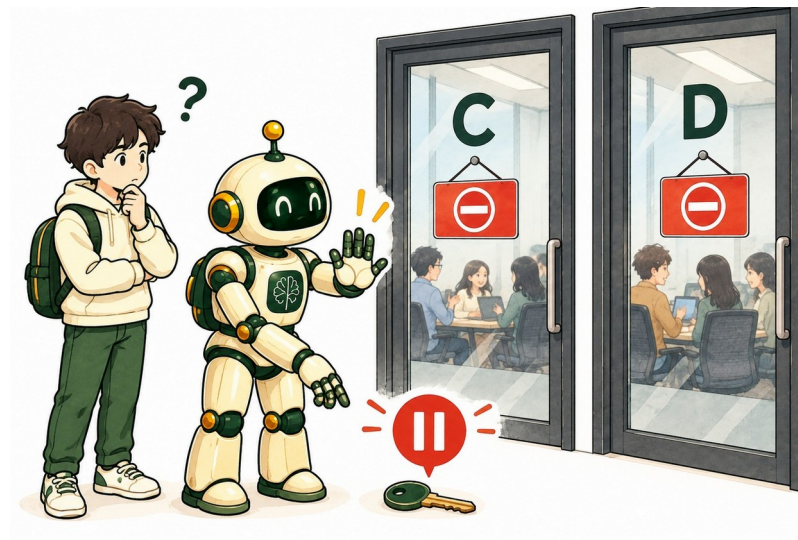
- ☒ Find suitable rooms
- ☒ Check their times
- ☒ Book D: BK-1042
- ☐ Send invitations

No valid room? Stop and ask

C and D are both full.

Stop tool actions.

“Would another time work?”





香港大學
THE UNIVERSITY OF HONG KONG

AGENTS · LECTURE 5

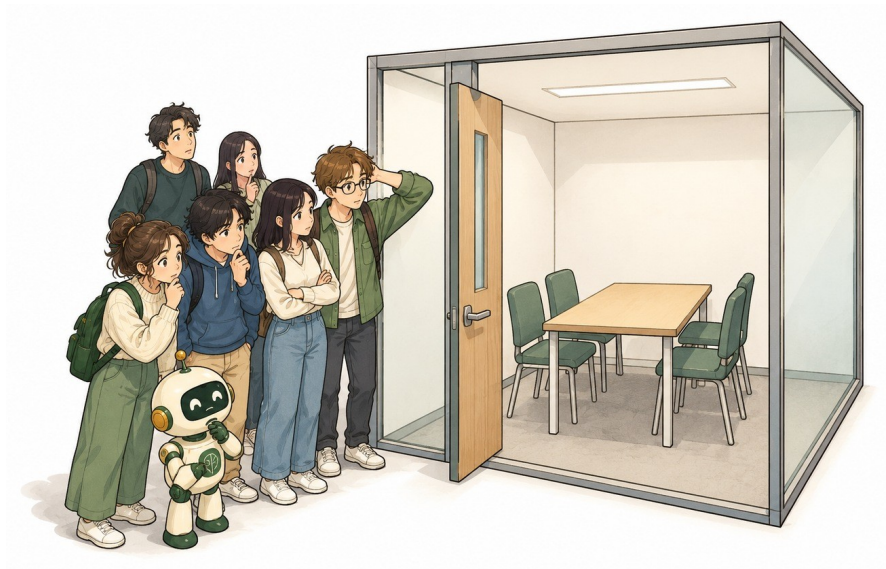
04

Memory

SO FAR

The plan changes as tools return
new facts.

PART 4 OF 7



Memory can begin with readable files

CLAUDE CODE

CLAUDE.md

memory/MEMORY.md

memory/topic-files.md

NANOBOT

USER.md

SOUL.md

memory/MEMORY.md

The application reads saved text into a later context.

One teaching assistant, one whole semester



What must the assistant remember when you chat?

Three useful memory horizons

Short-term

“Explain step 2.”

This conversation

Mid-term

“Continue my assignment.”

Ongoing work

Long-term

“Use examples first.”

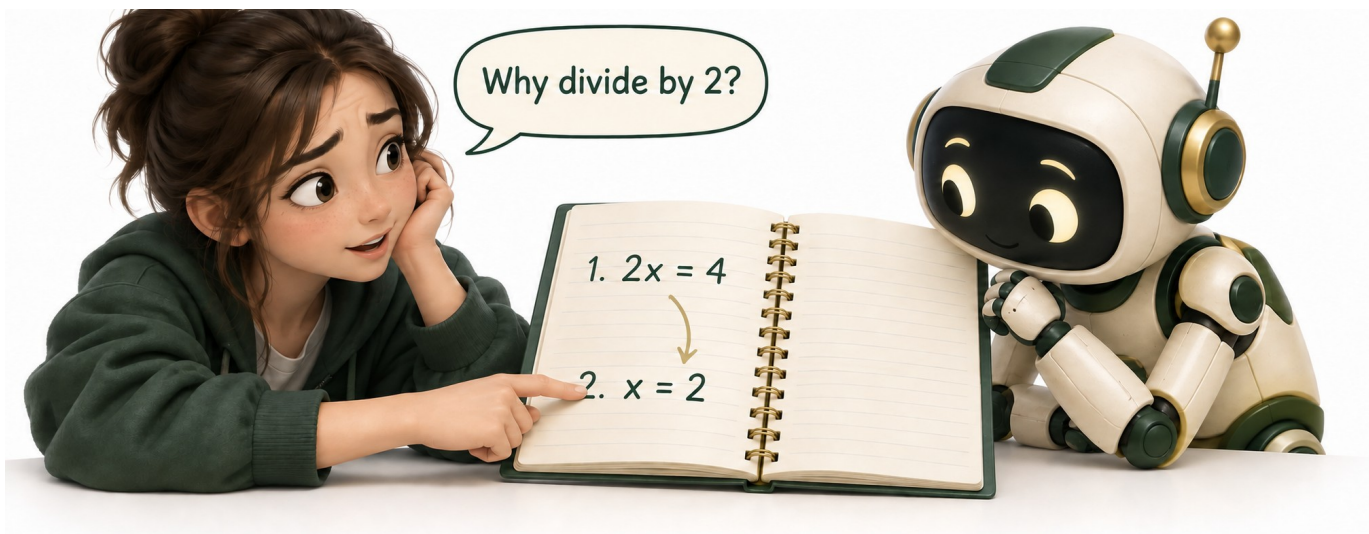
Stable preferences

Short-term: what does “step 2” mean?

Short-term

Mid-term

Long-term



The recent conversation tells us which step.

Mid-term: where did we stop?

Short-term

Mid-term

Long-term

ASSIGNMENT · HANDOFF SUMMARY

```
[x] Load the training data
[x] Fix the data leak
[ ] Rerun validation
[ ] Write the results section
```

Next: rerun validation on the saved split.

A short note helps the agent continue

Short-term

Mid-term

Long-term



WHAT TO KEEP

```
[x] Fix data leakage  
[ ] Run validation again  
[ ] Write the report
```

Continue from the next step.

Long-term: useful across many tasks

Short-term

Mid-term

Long-term

USER.md

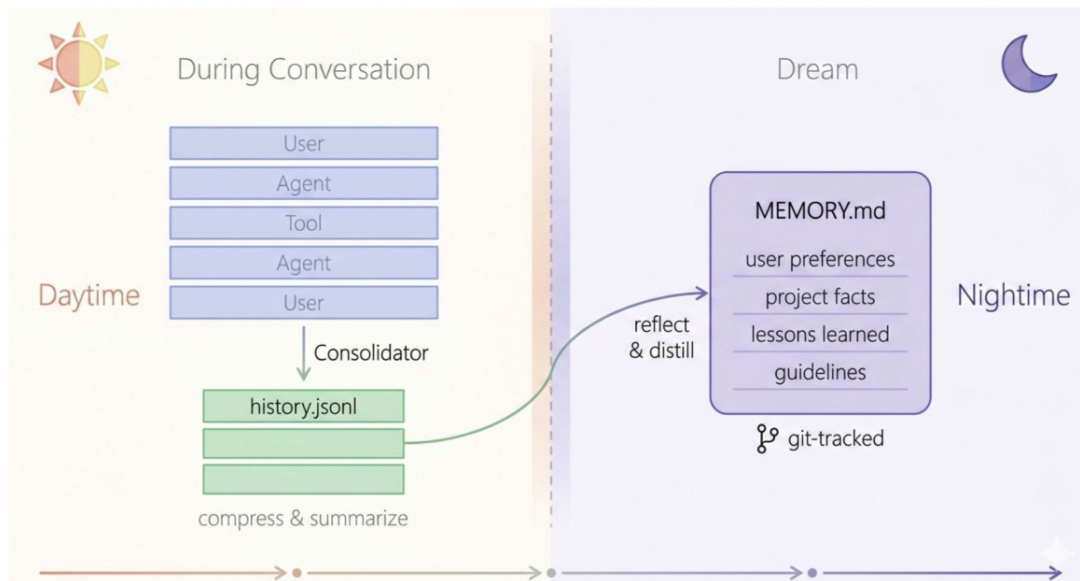
```
# Learning preferences
- Show a worked example
  before equations.
```

memory/MEMORY.md

```
# Course project
- Topic: retrieval
  evaluation
- Keep the data split.
```

NanoBot Dream: turning history into memory

Memory in nanobot



1 Save history

User + agent
+ tool messages

2 Dream

Keep useful facts
and preferences

The files have different jobs

<code>~/nanobot/</code>	
<code>sessions/<workspace-id>/*.jsonl</code>	conversations
<code>workspace/</code>	
<code>USER.md</code>	user preferences
<code>SOUL.md</code>	assistant style
<code>memory/</code>	
<code>history.jsonl</code>	compressed history
<code>MEMORY.md</code>	lasting knowledge

A corrected fact should replace an old one

OLD NOTE

Demo: 20 Oct

VERIFIED UPDATE

Demo: 27 Oct



```
source: course announcement  
updated: 12 Oct  
replaces: 20 Oct
```

Long memory: find the notes we need

“Why did run_07 fail?”

A semester of history

Week 1: prefer worked examples

Week 3: project uses PyTorch

run_07 stopped: GPU memory full

Week 5: demo moved to 27 Oct

Search



This call's context

run_07

**Stopped because
GPU memory was
full.**

NanoBot: load the summary. Search the history when needed.

Text storage does not decide how we search

The same text can support different search methods.

Keyword

Match exact words

run_07

Best for IDs and dates

Embedding

Match similar meaning

“ran out of memory”
≈ “GPU memory full”

Useful when wording differs

Hybrid

Use both signals

Exact ID + related
meaning

Then check the original note

NanoBot uses text search here. Embedding and hybrid are possible extensions.

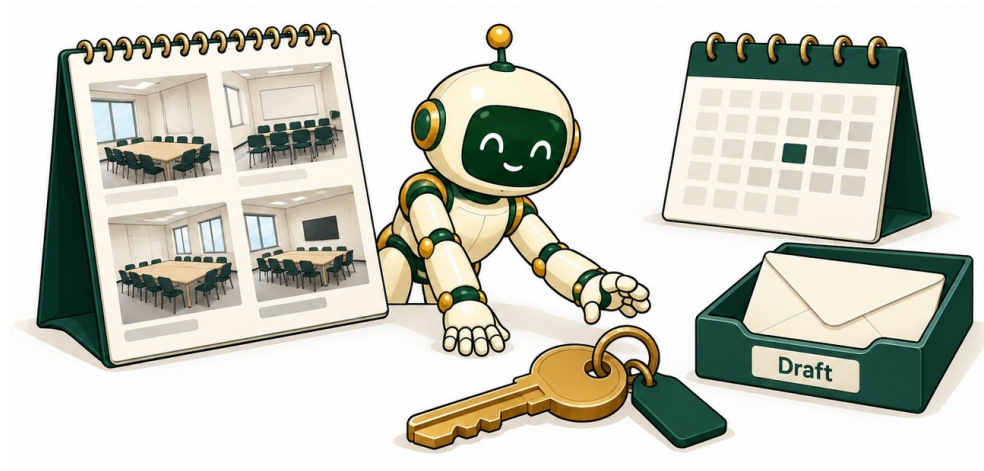


05

Tool-use

SO FAR

We can plan and preserve the job state.

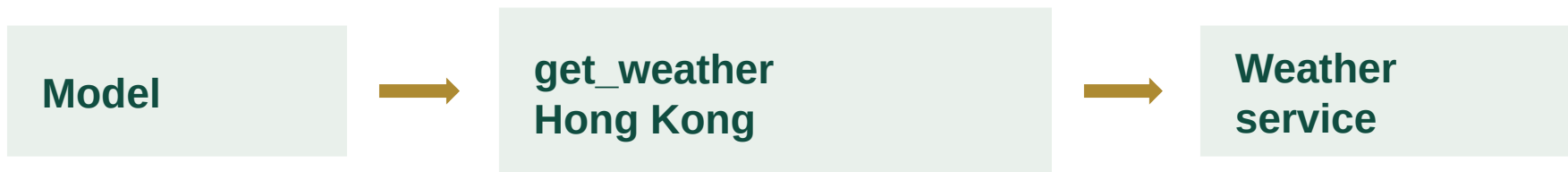


OUR NEXT QUESTION

How does a model decision become a reliable action?

Function calling: a precise request to a tool

“Is it raining in Hong Kong?”



The model asks. The application runs the function.

The tool definition describes valid inputs

```
{  
  "type": "function", "name": "get_weather",  
  "description": "Look up current weather in a city.",  
  "parameters": {  
    "type": "object",  
    "properties": {"location": {"type": "string"}},  
    "required": ["location"], "additionalProperties": false  
  },  
  "strict": true  
}
```

A function call is not the result yet

MODEL OUTPUT

```
{  
  "type": "function_call",  
  "call_id": "call_01",  
  "name": "get_weather",  
  "arguments": "{\\"location\\":\\"Hong Kong\\"}"  
}
```

Application: parse arguments, then run `get_weather`.

The application returns the observation

```
{  
  "type": "function_call_output",  
  "call_id": "call_01",  
  "output": "{\\"condition\\":\\"rain\\"}"  
}
```

Now the model can answer using the tool's data.

What function calling does and does not check

VALID FORMAT

```
{"location": "HONG GONG"}
```

Wrong city

A valid request can still be the wrong request.

Independent lookups call can run together

“Check the weather in Hong Kong, Tokyo and London.”



One request can launch several independent calls.

Why a common tool protocol?



Each service brings its own connection details.

Why MCP after function calling?

Function calling

The model asks for an action.

Tool name + arguments

“List the files in this folder.”



MCP: Model Context Protocol

The app still needs a way to reach the tool.

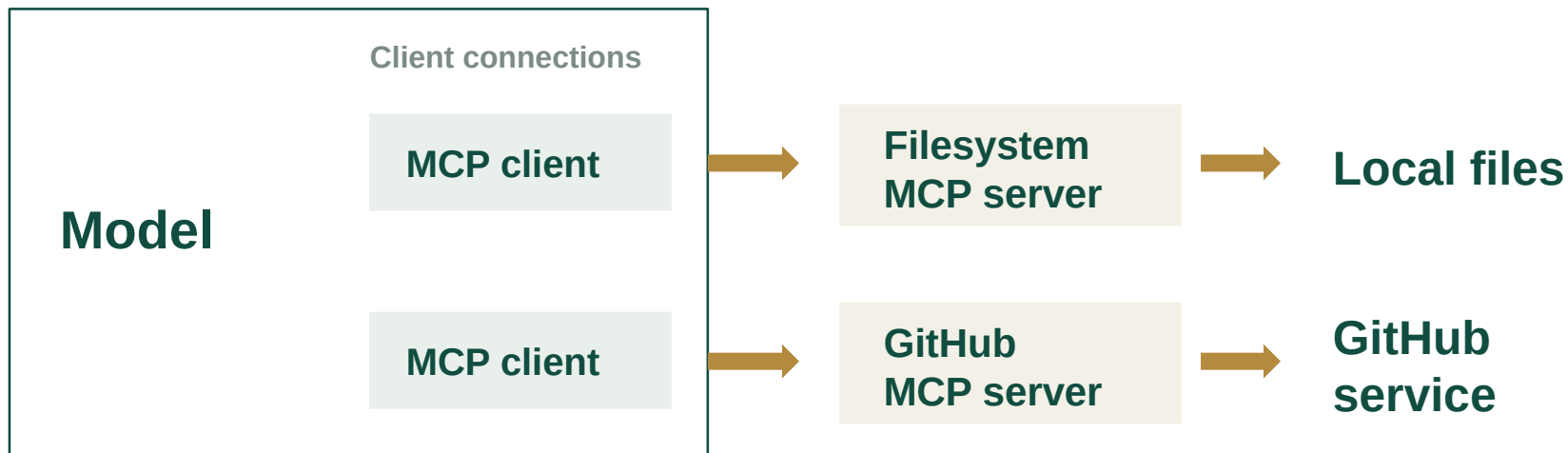
Discover tools
Call tools and read results

A shared protocol reduces repeated connection work.

One app can connect to several MCP servers

HOST: AI application

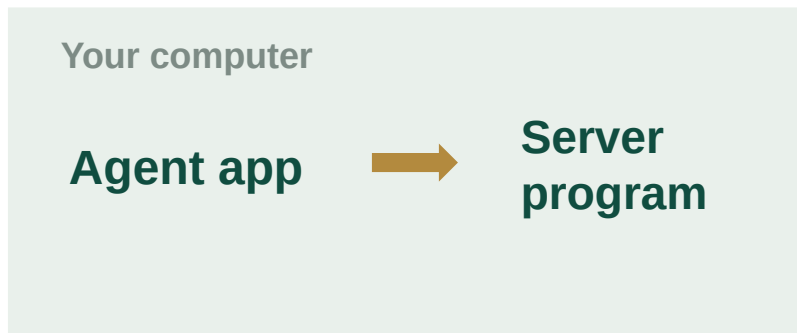
TOOL PROVIDERS



Each client manages one server connection.

Must we download an MCP server?

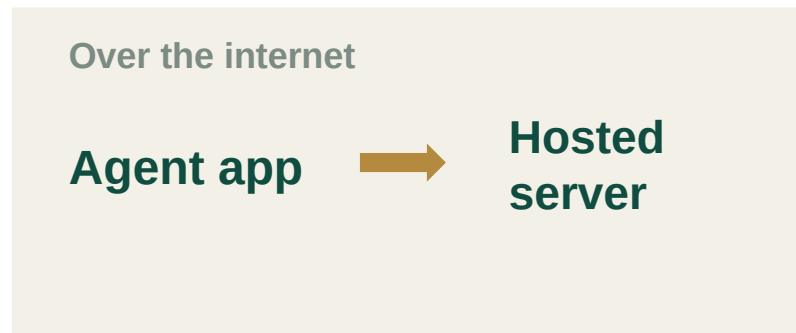
Local server



Install / run the program.

Example: local filesystem access

Remote server



Add the URL and sign in.

Example: hosted GitHub MCP

MCP: a shared protocol between app and server

AI application

MCP client

MCP server

Provides tools



Tool names, inputs and backend APIs can still differ.

MCP sends a call to the server

MCP CLIENT → MCP SERVER

```
{  
  "jsonrpc": "2.0", "id": 2,  
  "method": "tools/call",  
  "params": {  
    "name": "calculate_sum",  
    "arguments": {"a": 18.5, "b": 6.5}  
  }  
}
```

MCP returns the result in a common format

MCP SERVER → MCP CLIENT

```
{
  "jsonrpc": "2.0", "id": 2,
  "result": {
    "content": [{"type": "text", "text": "25"}],
    "isError": false
  }
}
```

CLI: instructions typed for a program

CLI = Command-Line Interface

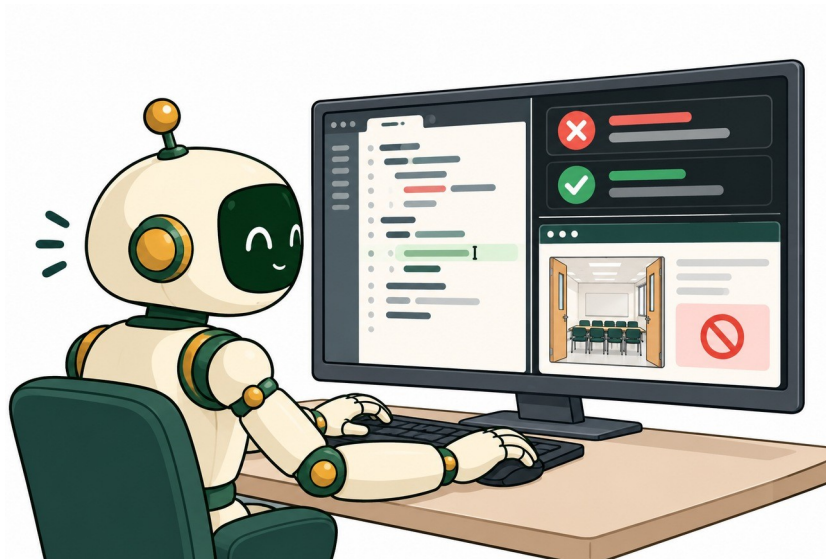
“Show open code reviews.”

1 Run a command

```
gh pr list
```

2 Read the result

```
#12 Fix login bug  
#18 Update the guide
```



A real CLI command

```
gh pr list --repo HKUDS/OpenSpace \  
  --state open --limit 5 \  
  --json number,title,url
```

Program + command + options

CLI output gives the agent something to check

STANDARD OUTPUT · EXAMPLE SHAPE

```
[  
  {"number": 12, "title": "Fix a test",  
   "url": "https://github.com/owner/repo/pull/12"}  
]
```

Check the exit code, output and errors.

Source: GitHub CLI · illustrative output

Why do we need Skills?

Why explain the same workflow every time?



Write it once. Read, update and share it.

People can read it. Compatible agents can reuse it.

Source: Agent Skills specification · teaching illustration

A PPT Skill is a guide with useful files

```
ppt-maker/  
  SKILL.md  
  assets/  
    template.pptx  
  references/  
    layout.md  
  scripts/  
    check_slides.py
```

Instructions

How to make and check slides

Supporting files

Template, layout rules, helpers

Inside the PPT Skill

name: ppt-maker

description: Create and edit PowerPoint slides.

Workflow

1. Read the template and plan the story.
2. Build each slide around one main idea.
3. Render every slide and inspect the images.
4. Fix overlaps, then export PPTX and PDF.

The PPT Skill loads in stages

1 Find the Skill

`ppt-maker`
`Make slides`



2 Read the guide

`SKILL.md`

`Build → render`
`→ inspect → fix`



3 Load files

`layout.md`
`template.pptx`
`check_slides.py`

More detail enters the context as the task needs it.

One PPT task can use several tools

SKILL.md: build, render, inspect, fix

1

Read the template

```
read_file(...)
```

2

Build the deck

```
run build script
```

3

Check the preview

```
view_image(...)
```

The application can run a CLI or call an MCP service.

Four concepts, four different jobs

Function call

A structured request from the model

Tool name + arguments. The app executes it.

MCP

A protocol between apps and tool providers

Discover tools, call them and receive results.

CLI

A program's text-command interface

Run a command. Read its output and errors.

Skill

A reusable guide for a type of task

Instructions, with optional scripts and resources.

They solve different parts of the same job.

Source: Function calling / MCP / GitHub CLI / Agent Skills specification



香港大學
THE UNIVERSITY OF HONG KONG

AGENTS · LECTURE 5

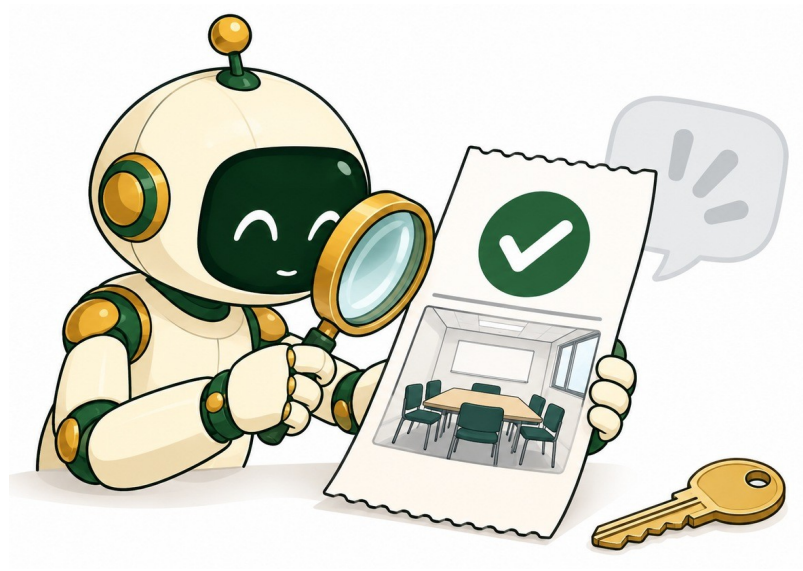
06

Insights

SO FAR

The agent has planned,
remembered and used tools.

PART 6 OF 7



OUR NEXT QUESTION

**What becomes harder as tasks
get longer?**

Model training is a long-horizon task

Recall Lecture 2: train a small language model



How should the agent plan across many experiments?

How should an agent plan a long task?

Example: train a small model within a fixed budget



The context is full. How can the agent continue?



Keep the goal, key decisions and next step in context.

Store full logs outside it. Load details when needed.

Source: LangChain / Anthropic / interview discussions · classroom example

What should survive into the next session?

MEMORY.md

For the agent

```
[x] Prepare train / val data
[x] Run the first experiment
[ ] Inspect rising val loss
[ ] Choose the next run
```

checkpoint.pt

For the training program

Model weights
Optimizer state
Training step

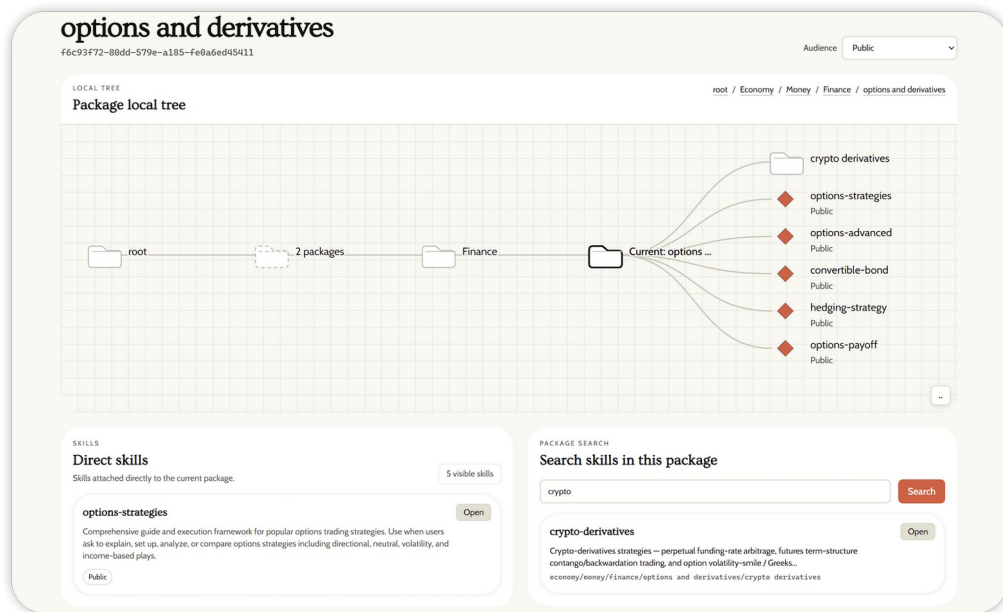
Open question: how much task history does the agent need?

Which Skill should make this PPT?



A large Skill library needs a way to find the right guide.

OpenSpace organizes Skills in a searchable library



Finding a Skill for the PPT task

“Create lecture slides”

ppt-maker
slide-design
pdf-export
image-search
...

BM25 candidates



Embedding rerank

ppt-maker

Read its SKILL.md

Candidate names are illustrative.

The selected guide must enter the context

```
search_skills(query="create lecture slides")
```

Select / import



Read SKILL.md



Apply

```
read_file("ppt-maker/SKILL.md")
```

Check the source and permissions before using a new Skill.

Did the PPT Skill help?

OUTPUT FILES

```
lecture.pptx  
lecture.pdf  
slide previews/
```

CHECKED RESULTS

- ☐ Files open
- ☐ No overlaps
- ☐ Images fit the topic

Record the result together with the Skill version.

Skill Evolution: improve the guide after a failed slide

The preview reveals a problem

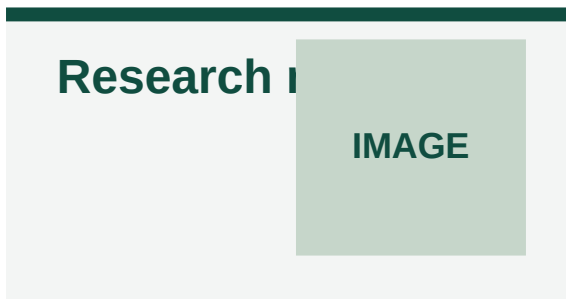


Image covers the title

Candidate update to SKILL.md

```
## Layout checks  
+ Keep images below the title.  
+ Re-render after each fix.
```

Does the revised Skill also help on another deck?

OpenSpace supports three kinds of Skill updates

FIX

Repair the current guide

ppt-maker v1



ppt-maker v2

DERIVED

Adapt a guide for a new task

ppt-maker



research-talk

CAPTURED

From scratch

Verified layout
check



new layout-check
Skill



香港大學
THE UNIVERSITY OF HONG KONG

AGENTS · LECTURE 5

07

Assignment

SO FAR

The loop acts, inspects results and improves.

PART 7 OF 7



OUR NEXT QUESTION

Can your policy improve through play?

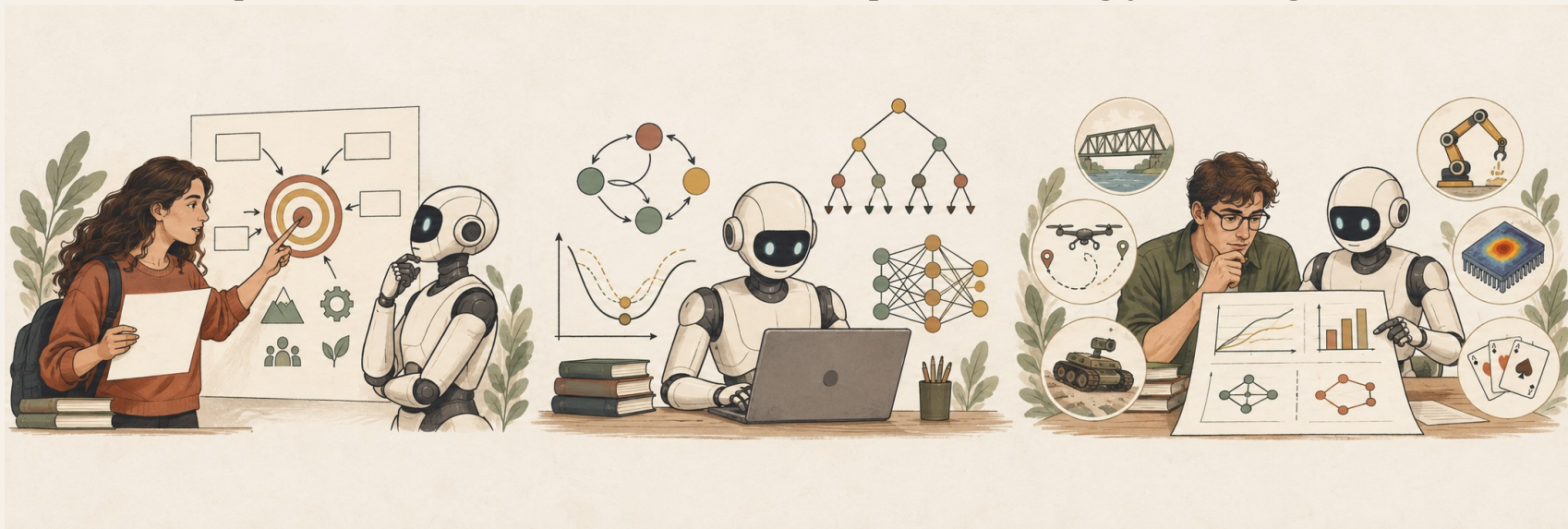
Featured Competitions

Multiplayer Texas Hold'em

Multiplayer Tank Battle

Open Methods and AI-native Development

Methods are open within task limits. AI-native development is strongly encouraged.



You define the goal

Set goals and constraints.
Describe your approach to AI.

AI develops the code

Ask AI to write, debug
and revise your strategy code.

You test and improve

Review results and replays.
Give AI feedback to improve.

Texas Hold'em: Rules

Six players. Hidden cards. Decisions under uncertainty.



2 private cards + 5 shared cards

Make the best five-card hand.

Your policy sees only its own cards.

Each turn

Fold, check, call, bet, raise or go all-in.

Win the pot with the best hand,
or make everyone else fold.

How this match ranks players

240 hands with seat rotation.

Fresh 10,000-chip stacks each hand.

Higher cumulative net returns win.

Preflop ➔ Flop ➔ Turn ➔ River ➔ Showdown
 3 cards +1 card +1 card

Texas Hold'em: Hands and Winning

Best 5 out of 7: your 2 cards + 5 shared cards.

Hand ranks: strongest to weakest

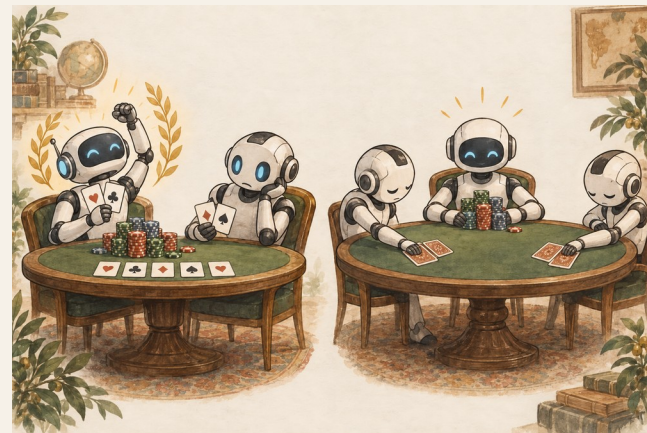
Straight flush	9♠ 8♠ 7♠ 6♠ 5♠
Four of a kind	Q♠ Q♥ Q♦ Q♣ 3♠
Full house	K♠ K♥ K♦ 2♣ 2♥
Flush	A♥ J♥ 8♥ 6♥ 2♥
Straight	9♠ 8♥ 7♦ 6♣ 5♠
Three of a kind	7♠ 7♥ 7♦ K♣ 2♥
Two pair	J♠ J♥ 4♦ 4♣ A♥
One pair	10♠ 10♥ K♦ 7♣ 2♥
High card	A♠ J♥ 9♦ 6♣ 3♥

A K Q J 10 of one suit is a royal flush.

Same type? Higher cards break ties.

Equal best hands split the pot.

Two ways to win the pot



1. Strongest hand

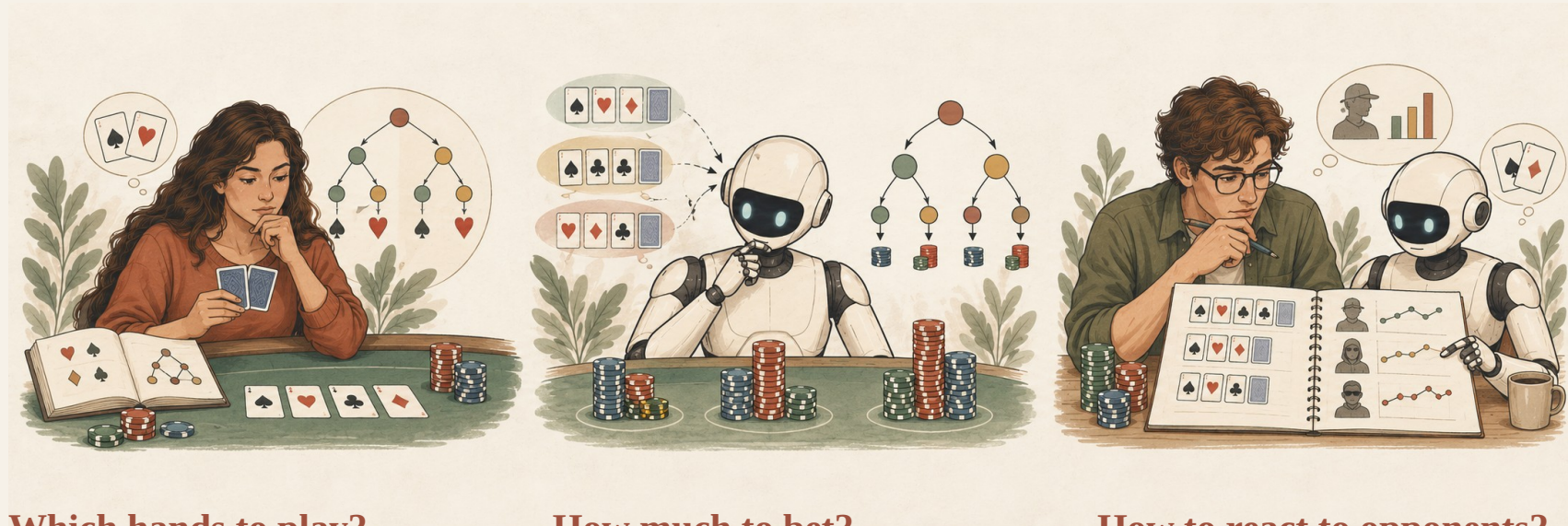
At showdown, the strongest remaining five-card hand wins the pot.

2. Everyone else folds

You win without showing your hand.

Texas Hold'em: Policy Design

Your policy chooses an action from the information it can observe.



Which hands to play?

Rules and decision trees can set entry thresholds.

How much to bet?

Monte Carlo simulation and search can compare risk and return.

How to react to opponents?

Opponent models, imitation learning and RL can adapt play.

You can combine methods. These are examples, not a required recipe.

Tank Battle: Rules

Four players. Three lives each. Up to two minutes per match.



Cover, visibility and survival change the best move.

What your tank can do

Move, aim, fire and use skills.
Use cover and collect supplies.
Enemies may be hidden from view.

What earns match points

- +3** per kill
- +2** per star
- +2** per remaining life

Total points determine match rank.

Tank Battle: Skill Choices

Pick one skill for the whole match. Your policy decides when to activate it.



Shield

Absorbs 2 damage
for up to 3 s.

Cooldown: 12 s

Boost

Faster movement.
Free turns for 4 s.

Cooldown: 10 s

Cloak

Hide beyond 1 tile
for 4 s. Firing ends it.

Cooldown: 14 s

Repair

Restore 1 HP now.
Health caps at 3 HP.

Cooldown: 15 s

Pierce

Next shot within 6 s:
2 damage; pierces one
brick wall or mound.

Cooldown: 11 s

Cooldown = time before you can use the skill again. Times shown are game time.

Tank Battle: Policy Design

The same situation can support several sensible strategies.



Collect a star, chase an enemy, or take cover?

Which goal comes first?

Rules or a finite-state machine can choose attack, collect or retreat.

Which route is safe?

Pathfinding, search and planning can navigate around obstacles.

When should you fire or use a skill?

Optimization, learned models or RL can compare timing and risk.

Any combination within the task limits.

AI-guided Policy Iteration

Your AI assistant can help improve both the policy and the way you improve it.



1. Set an objective

Tell AI what to improve.

2. Build a candidate

Ask AI to propose and edit.

3. Test and inspect

Use results and replays.

4. Diagnose and revise

Give AI concrete evidence.

Example: a tank gets stuck. AI proposes a route fix. Test whether it reduces failures.

Entry, Matches and Updates

5–11 October

Dates to be confirmed

Prepare offline and form teams.

1. Register



Sign in and register for a competition.

6. Revise and re-upload



Give AI feedback and test changes.
Re-upload at any time.

2. Develop with AI



Read the rules. Ask AI to develop your policy.

5. Rank and replay



Check your rank and replay.
Find useful evidence.

3. Upload your policy



Upload a JavaScript policy.

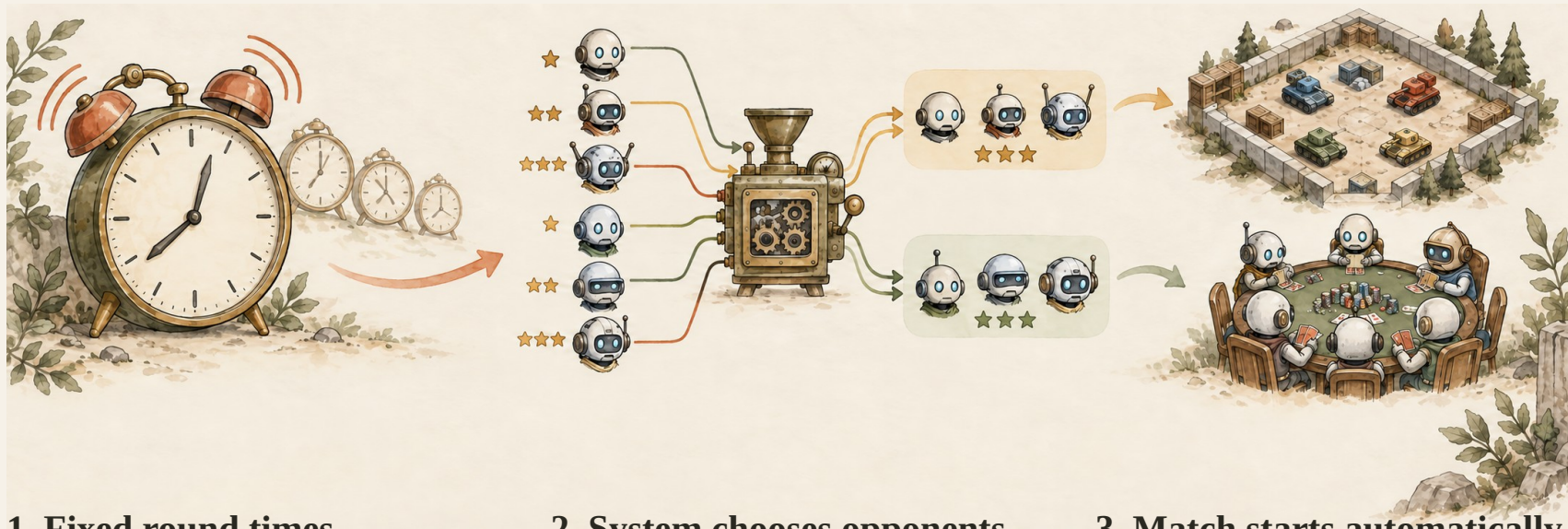
4. Automatic matches



Forced matching at fixed intervals.
Tank: 5 min. Poker: 10 min.

Automatic Matchmaking

Registered policies join forced matches at fixed times.



1. Fixed round times

Tank: every 5 minutes

Poker: every 10 minutes

2. System chooses opponents

Aims for similar estimated strength and fewer repeated opponents.

3. Match starts automatically

4 tanks or 6 poker players.
Your policy plays for you.

Still in an unfinished match? You skip that round.

Elo-style Rating

Match placement and opponent strength determine the rating update.

Both examples start at 1760

Beat a weaker opponent

Opponent rating: 1560



1760 to 1768

+8 rating points

Beat a stronger opponent

Opponent rating: 1960



1760 to 1777

+17 rating points

A win against stronger opposition usually earns more points.

Illustrative examples. Gains vary with the match and rating estimates.

Assessment and Team Requirements

The final report and rank determine your grade.



Algorithm and evidence

Explain your policy and support your choices with results.

Iteration and novelty

Show how it evolved, including any new RSI approach.

Teamwork

Explain how you collaborated and who contributed what.

Final standing

Report your final rank and interpret the outcome.

1–6 students per team

Solo teams are welcome.

One ladder account per team

Report cover: all members and the Google / GitHub account used on the ladder.

Assessment and Ranking

Final ranking

30%

+

Report

70%
